

SESSION 07

Tuples in Python

What is a Tuple in Python?

A tuple is a collection of values stored in a single variable, very similar to a list. The key difference is that a tuple is **immutable**. Once created, you cannot change, add, or remove its elements.

List – Can be Changed

```
cities_list = ["Delhi", "Mumbai", "Pune"]
cities_list[0] = "Chennai" # Allowed
print(cities_list) # Output: ['Chennai', 'Mumbai', 'Pune']
```

Tuple – Cannot be Changed

```
cities_tuple = ("Delhi", "Mumbai", "Pune")
cities_tuple[0] = "Chennai" # TypeError!
print(cities_tuple) # Output: TypeError!
```

Key properties of a tuple:

- **Ordered** — items stay in the order you put them.
- **Immutable** — cannot be modified after creation.
- **Allows duplicates** — the same value can appear more than once.
- **Can hold any data type** — integers, strings, booleans, even lists.

Tuples are used when you want to store data that should not change, such as coordinates, RGB colour values, or database records where the structure or content should remain constant.

Creating a Tuple

Tuples are created using parentheses `()` instead of square brackets `[]`. Let's look at some examples:

```
# Empty tuple
empty = ()

# Tuple of integers
marks = (85, 90, 78, 92, 88)

# Tuple of strings
cities = ("Delhi", "Mumbai", "Pune", "Chennai")

# Tuple of mixed types
profile = ("Rahul", 21, 9.2, True)

# Tuple without parentheses — also valid, known as "tuple packing"
point = 10, 20, 30
print(type(point)) # <class 'tuple'>
```

Single element tuple – the comma trick

Creating a tuple with a single element requires a special syntax. Without a trailing comma, Python treats parentheses as mere grouping.

```
# This is NOT a tuple — it's just a string in parentheses
not_a_tuple = ("hello")
print(type(not_a_tuple)) # <class 'str'>

# This IS a tuple — note the comma
actual_tuple = ("hello",)
print(type(actual_tuple)) # <class 'tuple'>
```



Watch Out: For a single-element tuple, the trailing comma is mandatory. Without it, Python treats the parentheses as just grouping, and you do not get a tuple.

Indexing and Slicing Tuples

Just like lists, tuples in Python support both indexing and slicing. These operations allow you to access individual elements or portions of a tuple efficiently.

Accessing Elements (Indexing)

Positive indexing starts from 0 for the first element, while negative indexing starts from -1 for the last element.

```
fruits = ("apple", "mango", "banana", "grapes", "orange")  
#      0      1      2      3      4  
#     -5     -4     -3     -2     -1
```

Positive indexing

```
print(fruits[0]) # Output: apple  
print(fruits[3]) # Output: grapes
```

Negative indexing

```
print(fruits[-1]) # Output: orange  
print(fruits[-4]) # Output: mango
```

Extracting Portions (Slicing)

Slicing allows you to extract a sub-tuple using start:stop:step syntax. It works identically to list slicing.

```
nums = (10, 20, 30, 40, 50, 60, 70, 80)  
  
print(nums[1:5]) # Output: (20, 30, 40, 50)  
print(nums[:4]) # Output: (10, 20, 30, 40)  
print(nums[4:]) # Output: (50, 60, 70, 80)  
print(nums[::2]) # Output: (10, 30, 50, 70)  
print(nums[::-1]) # Output: (80, 70, 60, 50, 40, 30, 20, 10)
```

 **Important Note:** Slicing a tuple always returns a **new tuple**, not a list.

Immutability: What You Can't Change

Immutability means that once a tuple is created, its size and the values of its elements cannot be altered. You cannot update an existing element, add a new one, or remove any of its items.

```
python_colors = ("red", "green", "blue")

# All of these will raise TypeError:
# python_colors[0] = "yellow"      # TypeError: 'tuple' object does not support item assignment
# python_colors.append("purple")   # AttributeError: 'tuple' object has no attribute 'append'
# del python_colors[1]             # TypeError: 'tuple' object doesn't support item deletion
```

However, it's crucial to understand that while the tuple itself is immutable, it can contain references to mutable objects, such as lists. The reference to the mutable object stored in the tuple cannot change, but the mutable object itself can still be modified.

```
python_data = ("Rahul", 21, [85, 90, 78])

# You cannot change which list is inside the tuple:
# python_data[2] = [100, 100, 100]  # TypeError: 'tuple' object does not support item assignment

# But you CAN modify the list itself, because the list is mutable:
python_data[2].append(95)
print(python_data)  # Output: ('Rahul', 21, [85, 90, 78, 95])
```

❏ **Quick Tip:** The tuple itself is immutable, guaranteeing that the references it holds won't change. But if it holds a mutable object (like a list), that object's internal state can still be modified.

Tuple vs. List: Choosing the Right Collection

Understanding the differences between tuples and lists is fundamental for efficient Python programming. While both are used to store collections of data, their mutability defines their primary use cases.

Feature	List	Tuple
Syntax	[1, 2, 3]	(1, 2, 3)
Mutability	Mutable (can change)	Immutable (cannot change)
Methods Available	Many (append, remove, sort, etc.)	Only 2 (count, index)
Performance	Slightly slower	Slightly faster
Memory Usage	More memory	Less memory
Use When	Data will change over time	Data should not change

When to use a List:

- Collection of items that will grow or shrink dynamically.
- You need to sort, insert, or remove items frequently.
- Examples: shopping cart, list of students, search results.

Here are some practical examples:

```
# List — items change over time
cart = ["apple", "milk"]
cart.append("bread")
print(cart) # Output: ['apple', 'milk', 'bread']

# Tuple — fixed data that should not change
rgb_red = (255, 0, 0)
coordinates = (28.6139, 77.2090) # Delhi lat/long
print(rgb_red) # Output: (255, 0, 0)
print(coordinates) # Output: (28.6139, 77.209)
```

When to use a Tuple:

- Data that should never change after creation.
- You want to protect data from accidental modification.
- Examples: coordinates (x, y), RGB colours, database records, days of the week.

Built-in Functions on Tuples

Tuples, like lists, can leverage many of Python's powerful built-in functions for common operations. These functions help you quickly analyse and manipulate your tuple data.

Basic Aggregate Functions

Functions like `len()`, `min()`, `max()`, and `sum()` work exactly the same way with tuples as they do with lists, providing quick insights into your data.

```
python_marks = (85, 92, 78, 95, 60, 88)

print(len(python_marks)) # 6    — number of elements
print(min(python_marks)) # 60   — smallest value
print(max(python_marks)) # 95   — largest value
print(sum(python_marks)) # 498  — sum of all elements
```

Sorting Tuples

The `sorted()` function can sort the elements of a tuple. However, it's crucial to remember that `sorted()` always returns a new **list**, not a tuple.

```
nums = (40, 10, 30, 20, 50)
result = sorted(nums)
print(result)      # [10, 20, 30, 40, 50]
print(type(result)) # <class 'list'>
```

If you need the sorted result as a tuple, you can convert the list back into a tuple using the `tuple()` constructor:

```
sorted_tuple = tuple(sorted(nums))
print(sorted_tuple) # (10, 20, 30, 40, 50)
```

Converting Between Lists and Tuples

Python provides convenient ways to convert between lists and tuples using their respective constructor functions:

```
my_list = [1, 2, 3, 4]
my_tuple = tuple(my_list) # (1, 2, 3, 4)

back_to_list = list(my_tuple) # [1, 2, 3, 4]
```



Important Note: The `sorted()` function always returns a **list**, regardless of whether its input was a list or a tuple. If a tuple output is desired, an explicit conversion using `tuple()` is required.

Tuple Methods

Since tuples are immutable, they provide only two built-in methods, both used for reading data rather than modifying it.

count(): Frequency of Elements

The `count()` method efficiently returns the number of times a specified value occurs within the tuple.

```
numbers = (10, 20, 30, 20, 40, 20, 50)
```

```
print(numbers.count(20)) # 3
```

```
print(numbers.count(100)) # 0
```

index(): Locating Elements

The `index()` method returns the index of the first occurrence of a specified value. If the value is not found, a `ValueError` is raised.

```
numbers = (10, 20, 30, 20, 40, 20, 50)
```

```
print(numbers.index(30)) # 2
```

```
print(numbers.index(20)) # 1 — only the first occurrence
```

```
# This line would raise an error:
```

```
# print(numbers.index(99)) # ValueError: tuple.index(x): x not in tuple
```

 **Quick Tip:** Unlike lists, which offer numerous methods for modification, tuples have only these two as they are designed to be unchangeable.

Looping Through a Tuple

Just like with lists, you can iterate over the elements of a tuple using various looping constructs in Python. The process is almost identical, allowing you to access each item sequentially.

Using a for loop

The for loop is the most common and Pythonic way to iterate through the elements of a tuple, directly accessing each item.

```
python_fruits = ("apple", "mango", "banana")

for fruit in python_fruits:
    print(fruit)
```

Using a while loop

You can also use a while loop, typically when you need more control over the iteration process or when dealing with an index.

```
python_fruits = ("apple", "mango", "banana")
i = 0

while i < len(python_fruits):
    print(python_fruits[i])
    i += 1
```

Using enumerate()

When you need both the index and the value of each element during iteration, the enumerate() function is incredibly useful. It adds a counter to an iterable and returns it as an enumerate object, which can then be used directly in a for loop.

```
python_cities = ("Delhi", "Mumbai", "Pune")

for index, city in enumerate(python_cities, start=1):
    print(f"{index}. {city}")

# Output:
# 1. Delhi
# 2. Mumbai
# 3. Pune
```

Tuple Packing and Unpacking

Tuple packing and unpacking are powerful features in Python that allow you to assign multiple values to a single variable (packing) or extract multiple values from a tuple into separate variables (unpacking). These operations streamline code and are fundamental to how tuples are used.

Packing: Grouping Values into a Tuple

Packing occurs when multiple values are assigned to a single variable without explicitly using parentheses. Python automatically groups these values into a tuple.

```
# Packing
person = "Rahul", 21, "Delhi"
print(person)      # Output: ('Rahul', 21, 'Delhi')
print(type(person)) # Output: <class 'tuple'>
```

Unpacking: Extracting Values from a Tuple

Unpacking is the reverse process, where elements of a tuple are assigned to individual variables. The number of variables on the left side of the assignment must exactly match the number of elements in the tuple.

```
person = ("Rahul", 21, "Delhi")

# Unpacking
name, age, city = person
print(name)      # Output: Rahul
print(age)       # Output: 21
print(city)      # Output: Delhi
```

 **Important:** If the number of variables does not match the number of elements in the tuple, Python will raise a `ValueError`.

```
point = (10, 20, 30)
# x, y = point    # This line would raise a ValueError: too many values to unpack
```

The "Swap Trick": Elegant Variable Swapping

Tuple unpacking provides an elegant and Pythonic way to swap the values of two variables in a single line, eliminating the need for a temporary variable.

```
a = 5
b = 10

# Traditional way
# temp = a
# a = b
# b = temp

# Tuple way — one line, no temp variable
a, b = b, a
print(a, b) # Output: 10 5
```

Returning Multiple Values from Functions

Functions in Python can conveniently return multiple values, which are automatically packed into a tuple. You can then unpack these values directly when calling the function.

```
def min_max(nums):
    return min(nums), max(nums) # returns a tuple

smallest, largest = min_max([4, 9, 2, 7, 1])
print(f"Min: {smallest}, Max: {largest}") # Output: Min: 1, Max: 9
```



Quick Tip: When a function returns multiple values, Python automatically packs them into a tuple. On the receiving end, you unpack them into separate variables. This mechanism is why tuples are used so frequently in Python for returning grouped results.

Practice Questions: Tuples

Time to test your knowledge! Work through these questions to solidify your understanding of tuples.

Q74: Create a tuple of 5 of your favourite songs. Print the first, last, and middle one using indexing.

Q75: Create a tuple of 8 numbers. Using slicing, print the first 3, last 3, and every alternate element.

Q76: Create a tuple of marks of 6 students. Print the highest, lowest, total, and average.

Q77: Take 5 numbers as input from the user, store them in a tuple, and print the tuple along with its minimum and maximum.

Q78: Write a function `get_stats(nums)` that takes a tuple of numbers and returns a tuple containing the sum, average, minimum, and maximum. Unpack the returned tuple and print each value.